

Timers and Intervals in JavaScript

JavaScript provides built-in functions to schedule code execution after a delay or at repeated intervals. These are useful for animations, delayed actions, auto-saving, and more.

`setTimeout()` : Run Code After a Delay

The `setTimeout()` function executes code **once** after a specified delay (in milliseconds).

Syntax

```
setTimeout(callback, delay);
```

Example

```
setTimeout(() => {  
  console.log('This runs after 2 seconds');  
}, 2000);
```

Canceling a Timeout

Use `clearTimeout()` to stop a scheduled timeout.

```
const timeoutId = setTimeout(() => {  
  console.log('Will not run');  
}, 3000);  
  
clearTimeout(timeoutId);
```

setInterval() : Run Code Repeatedly

The `setInterval()` function executes code **repeatedly** at a specified interval (in milliseconds).

Syntax

```
setInterval(callback, interval);
```

Example

```
setInterval(() => {  
  console.log('Runs every second');  
}, 1000);
```

Canceling an Interval

Use `clearInterval()` to stop the repetition.

```
const intervalId = setInterval(() => {  
  console.log('Repeating...');  
}, 1000);  
  
setTimeout(() => {  
  clearInterval(intervalId);  
  console.log('Interval stopped');  
}, 5000);
```

Use Cases

- Delayed popups or notifications
- Auto-refreshing data (like a clock or stock ticker)

- Game loops and animations
 - Debouncing or throttling user input
-

Important Notes

- Delays are **not guaranteed** to be exact — they depend on the event loop and execution stack.
 - Avoid overly frequent intervals (`<10ms`) as it may block the main thread.
-

Summary

Function	Purpose	Cancel with
<code>setTimeout()</code>	Run code once after delay	<code>clearTimeout()</code>
<code>setInterval()</code>	Run code repeatedly with delay	<code>clearInterval()</code>